

Reviewer Pack — Minimal Order of Formal Concept Analysis and Resolution Proo...

Entry 0ab9cc94de3b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 20:24:43 UTC

Minimal Order of Formal Concept Analysis and Resolution Proof Width Inequality

Entry ID: 0ab9cc94de3b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 20:24:43 UTC

1. Conjecture

Field A (mathematical branch): Formal Concept Analysis **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For a given Boolean formula φ in conjunctive normal form with m clauses, the minimal order of formal concepts associated with its clause set is linearly correlated with its resolution proof width, such that $m = \Theta(w(\varphi))$, where $w(\varphi)$ is the resolution proof width of φ .

Rationale (proposer's reasoning):

Formal Concept Analysis (FCA) provides a framework for understanding logical structures and their relationships. By mapping clauses of a Boolean formula to formal concepts, we can potentially uncover structural properties that influence its complexity. Resolution proof width is a well-studied measure of the difficulty of solving a SAT problem using the resolution method. The conjecture suggests that FCA could offer insights into this complexity measure by providing a direct link between the order of formal concepts and resolution proof width.

Taxonomy category: FCA_RESOLVER_WIDTH (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6cce0cefa91968ee

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 80% of the 30 random seeds, the correlation coefficient between the minimal order of formal concepts m and the resolution proof width $w(\varphi)$ is greater than or equal to 0.9, and no seed produces a correlation coefficient less than 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - formal concept analysis AND resolution proof complexity - resolution proof width IN formal concept analysis - Boolean satisfiability (resolution proof complexity) related to minimal order in formal concept analysis

Top relevant hits considered: - [<http://arxiv.org/abs/1210.2401v1>] Distributed Formal Concept Analysis Algorithms Based on an Iterative MapReduce Framework - [<http://arxiv.org/abs/1107.2822v1>] A Survey on how Description Logic Ontologies Benefit from Formal Concept Analysis - [<http://arxiv.org/abs/1809>] The Origins Space Telescope (OST) Mission Concept Study Interim Report - [<http://arxiv.org/abs/2012.09650v1>] A White Box Analysis of ColBERT - [<http://arxiv.org/abs/1103.5740v2>] Generating and Searching Families of FFT Algorithms

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(m):
        variables = set()
        clauses = []
        for _ in range(m):
            clause = []
            for _ in range(random.randint(1, 3)):
                var = f'x{random.randint(0, m)}'
                if var not in variables:
                    variables.add(var)
                clause.append(var)
            clauses.append(clause)
        return clauses

    def formal_concept_order(clauses):
        concepts = set()
        for clause in clauses:
```

```

        concept = tuple(sorted(clause))
        concepts.add(concept)
    return len(concepts)

def resolution_proof_width(clauses):
    # Simplified DPLL-based solver to estimate width
    stack = []
    literals = set()
    for clause in clauses:
        literals.update(clause)

    def dpll():
        if not stack:
            return 1
        literal = next(iter(literals))
        pos_literal, neg_literal = f'+{literal}', f'-{literal}'
        if pos_literal in literals and neg_literal in literals:
            literals.remove(pos_literal)
            literals.remove(neg_literal)
            stack.append((pos_literal, neg_literal))
            return dpll()
        elif pos_literal in literals:
            literals.remove(pos_literal)
            stack.append(pos_literal)
            return dpll()
        else:
            literals.remove(neg_literal)
            stack.append(neg_literal)
            return dpll()

    width = 0
    for _ in range(10): # Simplified sampling
        literals.clear()
        literals.update(clauses)
        width = max(width, len(stack))
    return width

n_trials = 30
m_values = [5, 10, 15, 20, 30, 40]
total_m = 0
total_w = 0

for _ in range(n_trials):
    m = random.choice(m_values)
    clauses = generate_formula(m)
    m_val = formal_concept_order(clauses)
    w_val = resolution_proof_width(clauses)
    total_m += m_val
    total_w += w_val

mean_m = Fraction(total_m, n_trials)
mean_w = Fraction(total_w, n_trials)

correlation_coefficient = (n_trials * sum(m_val * w_val for m_val, w_val in zip([mean_m] * n_trials, [mean_w] * n_trials)) -
                           total_m * total_w) / math.sqrt((n_trials * sum(m_val**2 for m_val in m_values) -
                                                             (n_trials * sum(m_val for m_val in m_values))**2) *
                                                             (n_trials * sum(w_val**2 for w_val in w_values) -
                                                             (n_trials * sum(w_val for w_val in w_values))**2))

```

```

conjecture_holds = correlation_coefficient >= 0.9
counterexample = "" if conjecture_holds else f"Correlation: {correlation_coefficient}"

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": float(correlation_coefficient),
    "instances_tested": n_trials,
    "n_max": max(m_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=NA support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Correlation below threshold\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_25357665.py", line 111, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_25357665.py", line 83, in run_trial
    w_val = resolution_proof_width(clauses)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_25357665.py", line 70, in resolution_proof_width
    literals.update(clauses)
TypeError: unhashable type: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to complete the required calculations for the conjecture. As a result, we cannot confirm whether the support condition of at least 80% correlation coefficient ≥ 0.9 is met. | next: Re-run the test with proper error handling and ensure that it completes without crashing to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18918
2	preregistration	ollama_remote	glm4:latest	0	0	14757
3	novelty	ollama_remote	glm4:latest	0	0	12088
4	novelty	ollama_remote	glm4:latest	0	0	12474
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13777
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10983
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11899
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11448
9	judge	ollama_remote	glm4:latest	0	0	14076

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120419 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/0ab9cc94de3b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/0ab9cc94de3b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0ab9cc94de3b.tar.gz (if generated)